

Microservice N Kafka

Explain Monolithic and Microservice
 Explain Microservice with Architecture
 Domain Driven Design Pattern
 Rest API calls for charge engine service or through producer n consumer rabbit mg
 Service Discovery Design Pattern and registration
 Config server

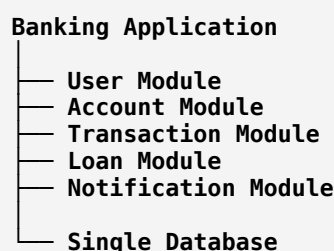
These are very important backend system design and Spring Boot interview topics. I'll explain them in a practical way with examples.

1. Monolithic Architecture

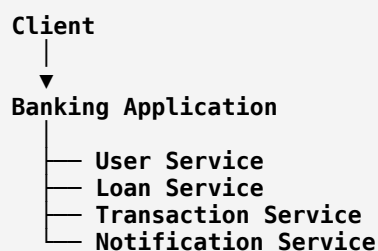
A monolithic application is a single deployable unit where all modules exist in one application.

Example

Banking Application



Flow



Everything runs inside one application.

Advantages

Easy Development

Single codebase.

Easy Deployment

Only one WAR/JAR.

Easy Testing

No service communication complexity.

Disadvantages

Large Codebase

As application grows, maintenance becomes difficult.

Single Point of Failure

If one module crashes:

```
Transaction Module Down
      ↓
Entire Application Down
```

Scalability Problem

Suppose only transaction module gets heavy traffic.

You must scale whole application.

```
Need more transaction processing
      ↓
Scale entire application
```

Costly.

2. Microservice Architecture

Application is divided into multiple small independent services.

Example

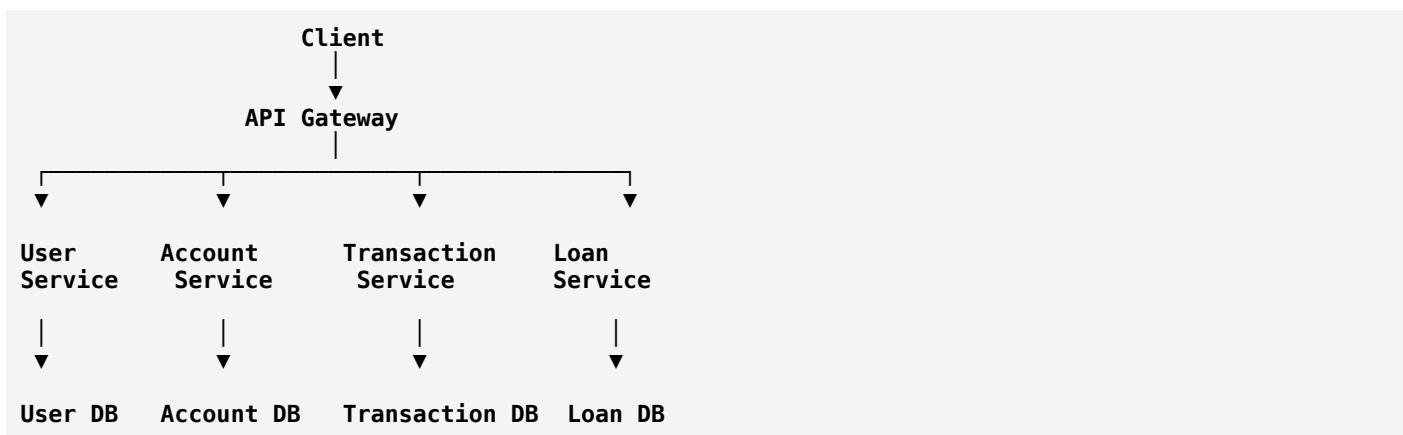
```
User Service
Account Service
Transaction Service
Loan Service
Notification Service
```

Each service:

- Independent
- Deployable separately

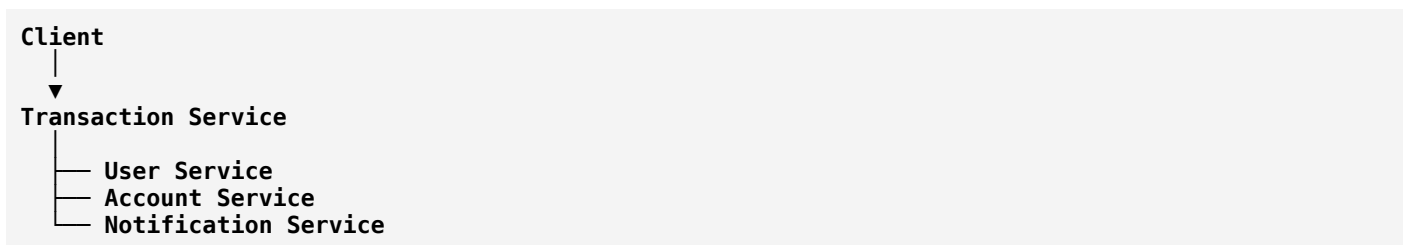
- Own database

Architecture



Real Example

When user transfers money:



Each service works independently.

Monolithic vs Microservices

Feature	Monolithic	Microservice
Deployment	Single	Multiple
Scaling	Entire App	Individual Service
Database	Shared	Separate
Maintenance	Difficult	Easy
Fault Isolation	Low	High
Development	Easy initially	Complex initially
Team Independence	Low	High

Interview Question:

Why companies move from monolith to microservices?

Answer:

- Better scalability
- Independent deployment
- Fault isolation
- Team autonomy
- Faster release cycles

3. Domain Driven Design (DDD)

DDD focuses on designing software around business domains.

Instead of thinking:

Tables
Controllers
Entities

Think:

Business
Rules
Processes

Example Banking System

Business Domains:

Customer Domain
Account Domain
Payment Domain
Loan Domain
Notification Domain

Each domain becomes a microservice.

DDD Architecture

Customer Service
Account Service
Loan Service
Payment Service
Notification Service

Each service owns:

Entity
Repository
Business Logic
Database

DDD Layers

Controller Layer

Service Layer

Domain Layer

Repository Layer

Database

Entity Example

```
Java
public class Account {

    private Long id;
    private String accountNumber;
    private double balance;

    public void withdraw(double amount) {
        if(balance < amount)
            throw new RuntimeException();

        balance -= amount;
    }
}
```

Business rule lives inside domain.

4. Inter-Service Communication

Microservices communicate using:

Option 1: REST API

Synchronous

```
Transaction Service
    |
    v
Account Service
    |
    v
Response
```

Example

```
Java
RestTemplate
WebClient
Feign Client
```

Feign Example:

```
Java
@FeignClient(name="account-service")
public interface AccountClient {

    @GetMapping("/account/{id}")
    Account getAccount(@PathVariable Long id);
}
```

Advantages

Simple
Easy debugging
Real-time response

Disadvantages

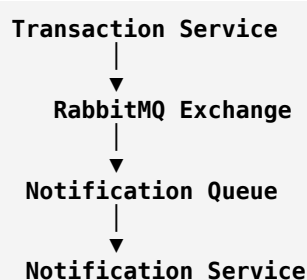
Tight coupling
Service dependency
Higher latency

Option 2: RabbitMQ Producer Consumer

Asynchronous Communication

Producer publishes message.
Consumer processes later.

Architecture



Example

After payment:

Transaction Completed

Send event:

```
Java
rabbitTemplate.convertAndSend(
    "payment.exchange",
    "payment.completed",
    event
);
```

Consumer:

```
Java
@RabbitListener(queues = "notification.queue")
public void consume(PaymentEvent event){
    sendSMS(event);
}
```

Advantages

- Loose Coupling
- High Performance
- Retry Support
- Event Driven

Disadvantages

- Complex Debugging
- Eventual Consistency

REST vs RabbitMQ

Feature	REST	RabbitMQ
Type	Sync	Async
Immediate Response	Yes	No
Coupling	Tight	Loose
Performance	Lower	Higher
Retry	Manual	Built-in
Best Use	Data Fetching	Notifications

Real Banking Example

Use REST for:

Get Customer Details
Get Account Balance
Validate User

Use RabbitMQ for:

Email Notification
SMS Notification
Audit Logs
Report Generation

5. Service Discovery Pattern

In microservices IP addresses change frequently.

Without service discovery:

Account Service
10.20.30.40

Tomorrow:
10.20.30.55

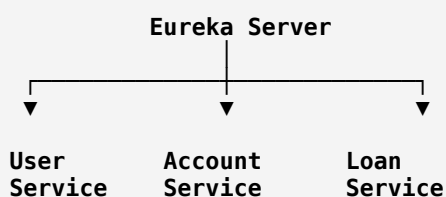
Other services fail.

Solution

Service Registry

All services register themselves.

Architecture



Registration

When Account Service starts:

Register me

Service Name:
ACCOUNT-SERVICE

IP:
10.20.30.40

Port:
8081

Stored in Eureka.

Service Lookup

Transaction Service asks:

```
Where is ACCOUNT-SERVICE?
```

Eureka replies:

```
10.20.30.40:8081
```

Spring Cloud Eureka

Server

```
Java
@EnableEurekaServer
```

Client

```
properties
eureka.client.service-url.defaultZone=
http://localhost:8761/eureka
```

6. Service Registration Flow

```
Account Service Starts
  |
  v
Register With Eureka
  |
  v
Heartbeat Every 30 Seconds
  |
  v
Eureka Keeps Service Alive
```

If service crashes:

```
No Heartbeat
  |
  v
Removed From Registry
```

7. Config Server

Problem:

Every microservice has configuration.

```
DB URL
Username
Password
```

RabbitMQ URL
Kafka URL

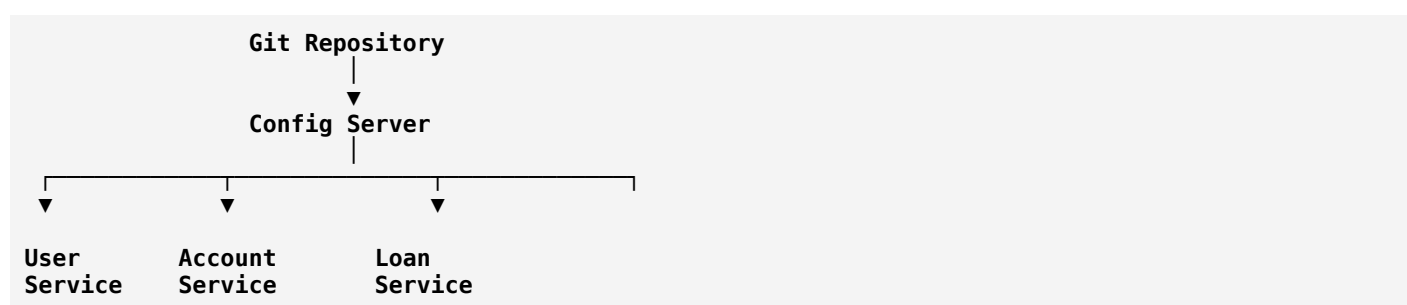
Managing them separately becomes difficult.

Solution

Centralized Configuration

Using Spring Cloud Config Server.

Architecture



Config Repository

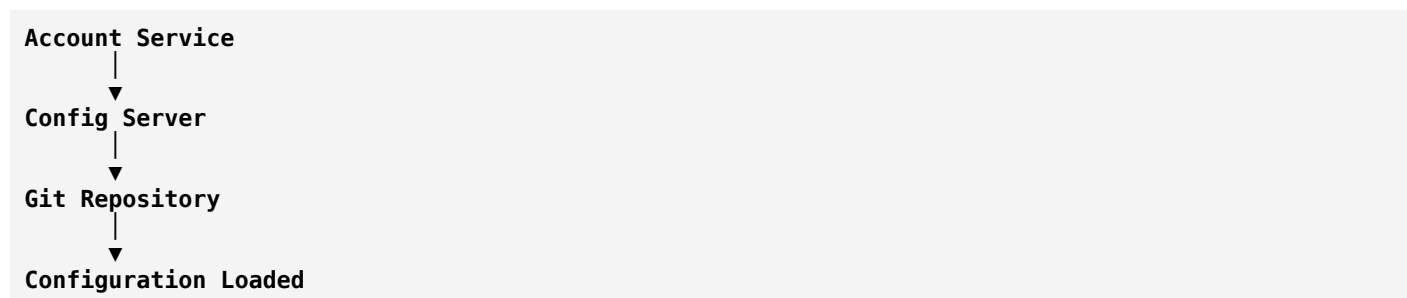
application.yml

```

YAML
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/bank
  
```

Stored once.

Service Startup



Benefits

Centralized Config

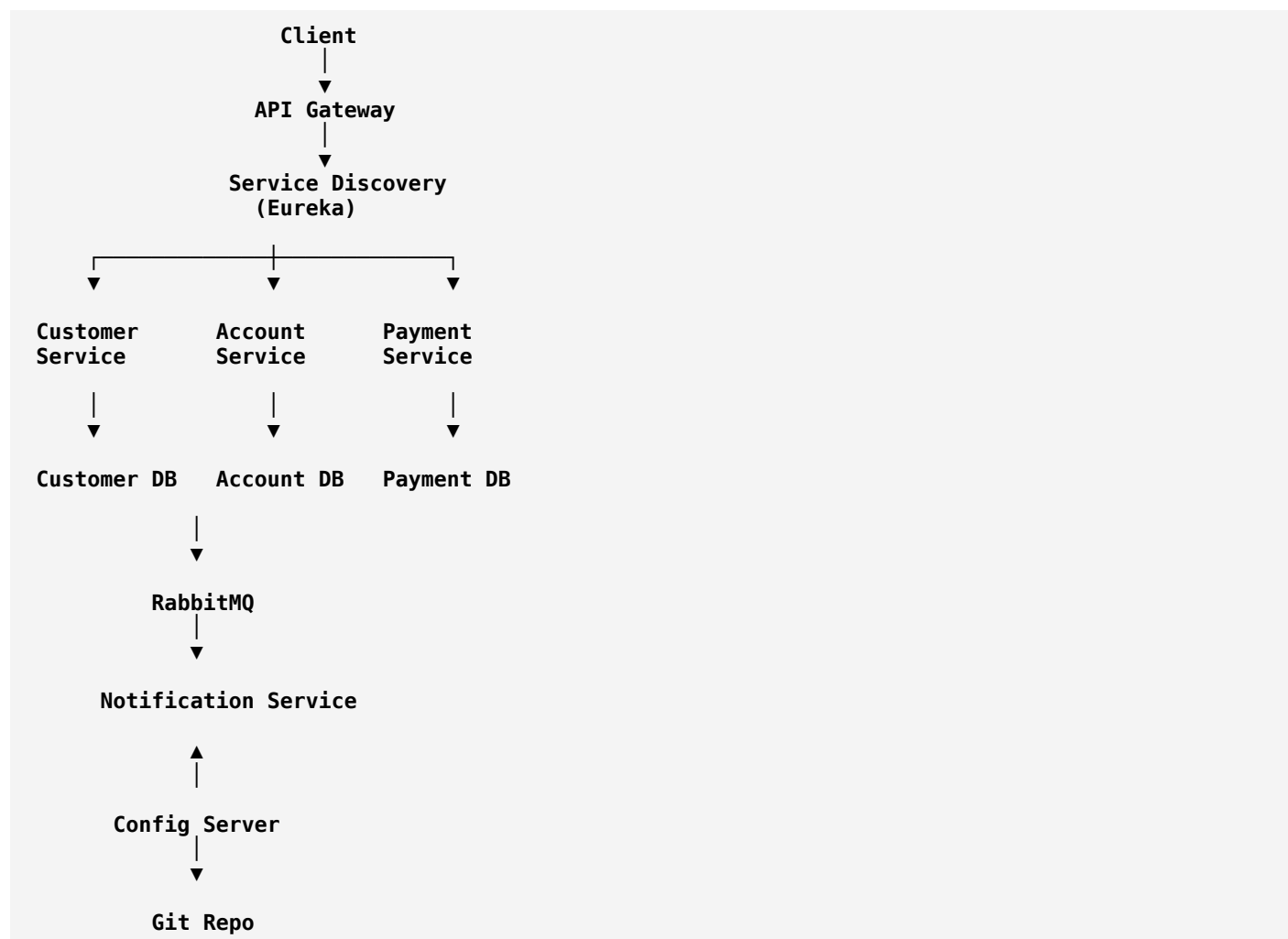
Version Control

Environment Specific Config

No Duplication

Easy Changes

Complete Microservice Architecture (Interview Diagram)



Frequently Asked Interview Questions

1. Why microservices over monolith?
2. What is DDD and bounded context?
3. Difference between REST and RabbitMQ?
4. Why use RabbitMQ if REST already exists?
5. What is service discovery?
6. How Eureka works internally?
7. What happens if Eureka server goes down?
8. Why Config Server?
9. Difference between API Gateway and Load Balancer?

10. What is eventual consistency in microservices?
11. How do microservices communicate securely?
12. How do you handle distributed transactions across services? (Saga Pattern)
13. What is circuit breaker pattern? (Resilience4j)
14. Why does each microservice have its own database?
15. What challenges did you face while working with microservices?

For a Spring Boot developer with 2 years of experience, also study **API Gateway, Feign Client, Circuit Breaker (Resilience4j), Saga Pattern, Distributed Tracing, Docker, Kubernetes, Kafka vs RabbitMQ, and OAuth2/JWT Security**, as these are commonly asked alongside microservices.

Explain how to configure microservices and how we can connect other microservice and how to handle failure if one of the service fails what should be the process

This is one of the most common 2-5 years Spring Boot interview topics. Let's go through how a real microservices system is configured and how failures are handled.

Example System

Suppose we have:

```
Customer Service
Account Service
Payment Service
Notification Service
```

When a payment is made:

```
Customer Service
  ↓
Payment Service
  ↓
Account Service
  ↓
Notification Service
```

1. How Microservices Are Configured

Each service is an independent Spring Boot application.

Customer Service

```
properties
spring.application.name=customer-service
server.port=8081
```

Account Service

```
properties
spring.application.name=account-service
server.port=8082
```

Payment Service

```
properties
spring.application.name=payment-service
server.port=8083
```

Each service:

- Own codebase
- Own database
- Own deployment

2. Service Registration with Eureka

Every service registers itself.

Eureka Server

```
Java
@EnableEurekaServer
@SpringBootApplication
public class EurekaServerApplication {
}
```

```
properties
server.port=8761
```

Client Service Configuration

```
properties
spring.application.name=account-service

eureka.client.service-url.defaultZone=
http://localhost:8761/eureka
```

When service starts:

```
Account Service
|
v
Register to Eureka
```

Eureka stores:

ACCOUNT-SERVICE
localhost:8082

3. How One Service Connects to Another

Method 1: Feign Client (Most Common)

Payment Service needs Account Service.

Dependency

XML
spring-cloud-starter-openfeign

Feign Client

```
Java
@FeignClient(name = "account-service")
public interface AccountClient {

    @GetMapping("/accounts/{id}")
    AccountDTO getAccount(
        @PathVariable Long id
    );
}
```

Usage

```
Java
@Service
public class PaymentService {

    @Autowired
    private AccountClient accountClient;

    public void processPayment() {

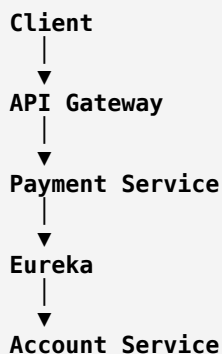
        AccountDTO account =
            accountClient.getAccount(1L);
    }
}
```

No need to know:

localhost
IP Address
Port

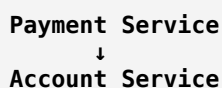
Eureka handles it.

4. How Request Flows



5. What If Account Service Fails?

Suppose:



Account Service is down.

Without protection:

Connection Refused
Exception
Payment Service Crash

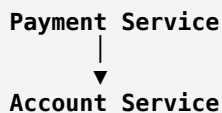
Bad design.

6. Use Circuit Breaker

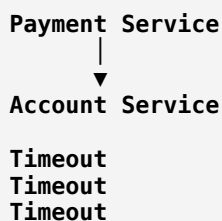
Most companies use:

Resilience4j

Normal Flow



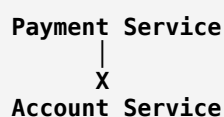
Service Down



Too many failures.

Circuit opens.

Open State



No calls sent.

Fallback executed immediately.

Example

```

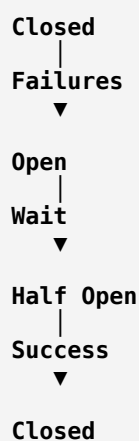
Java
@CircuitBreaker(
    name="accountService",
    fallbackMethod="fallback"
)
public AccountDTO getAccount(Long id){
    return accountClient.getAccount(id);
}
  
```

Fallback:

```

Java
public AccountDTO fallback(
    Long id,
    Exception ex){
    return new AccountDTO(
        id,
        "Service Unavailable"
    );
}
  
```

Circuit Breaker States



Interview Question:

Why Circuit Breaker?

Answer:

Prevents cascading failures and avoids unnecessary calls to failed services.

7. Retry Mechanism

Temporary failures happen.

Example:

```
Network issue
Database issue
Container restart
```

Retry before failing.

```
Java
@Retry(name = "accountRetry")
```

Configuration:

```
YAML
resilience4j:
  retry:
    instances:
      accountRetry:
        maxAttempts: 3
```

Flow:

```
Call 1
Failed

Call 2
Failed

Call 3
Success
```

8. Timeout Configuration

Never wait forever.

```
YAML
resilience4j:
  timelimiter:
    instances:
      accountService:
        timeoutDuration: 2s
```

After 2 seconds:

Timeout Exception
Fallback

9. Fallback Response

Instead of crashing:

```
Java
public AccountDTO fallback(
    Long id,
    Exception e){
    return cachedResponse();
}
```

Possible fallbacks:

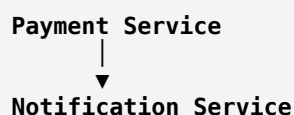
- Cached Data
- Default Response
- Error Message
- Queue Request For Later

10. Using RabbitMQ Instead of REST

For notifications:

Don't call Notification Service directly.

Bad:



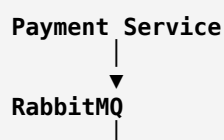
If notification service fails:

Payment Failed

even though money was transferred.

Better

Use message queue.



Notification Service

Producer:

```
Java
rabbitTemplate.convertAndSend(
    "payment.exchange",
    paymentEvent
);
```

Consumer:

```
Java
@RabbitListener(
    queues = "notification.queue"
)
public void consume(
    PaymentEvent event){
}
```

Benefits:

Loose Coupling
Retry
Persistence
Asynchronous

11. What If RabbitMQ Consumer Fails?

Message remains in queue.

```
RabbitMQ
|
v
Notification Service
Failed
```

RabbitMQ retries.

No message loss.

12. Database Failure Handling

Suppose Payment Service DB is down.

Use:

Connection Pool

Often via HikariCP

YAML

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 20
```

Retry

```
Java
@Retry
```

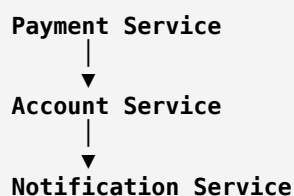
Monitoring

Use:

- Spring Boot Actuator
- Prometheus
- Grafana

13. Distributed Transaction Problem

Suppose:



Payment successful.

Notification failed.

Should we rollback payment?

In microservices:

✗ Avoid distributed database transactions (2PC).

Use:

Saga Pattern

Saga Pattern

```
Step 1:
Debit Account

Step 2:
Create Payment

Step 3:
Send Notification
```

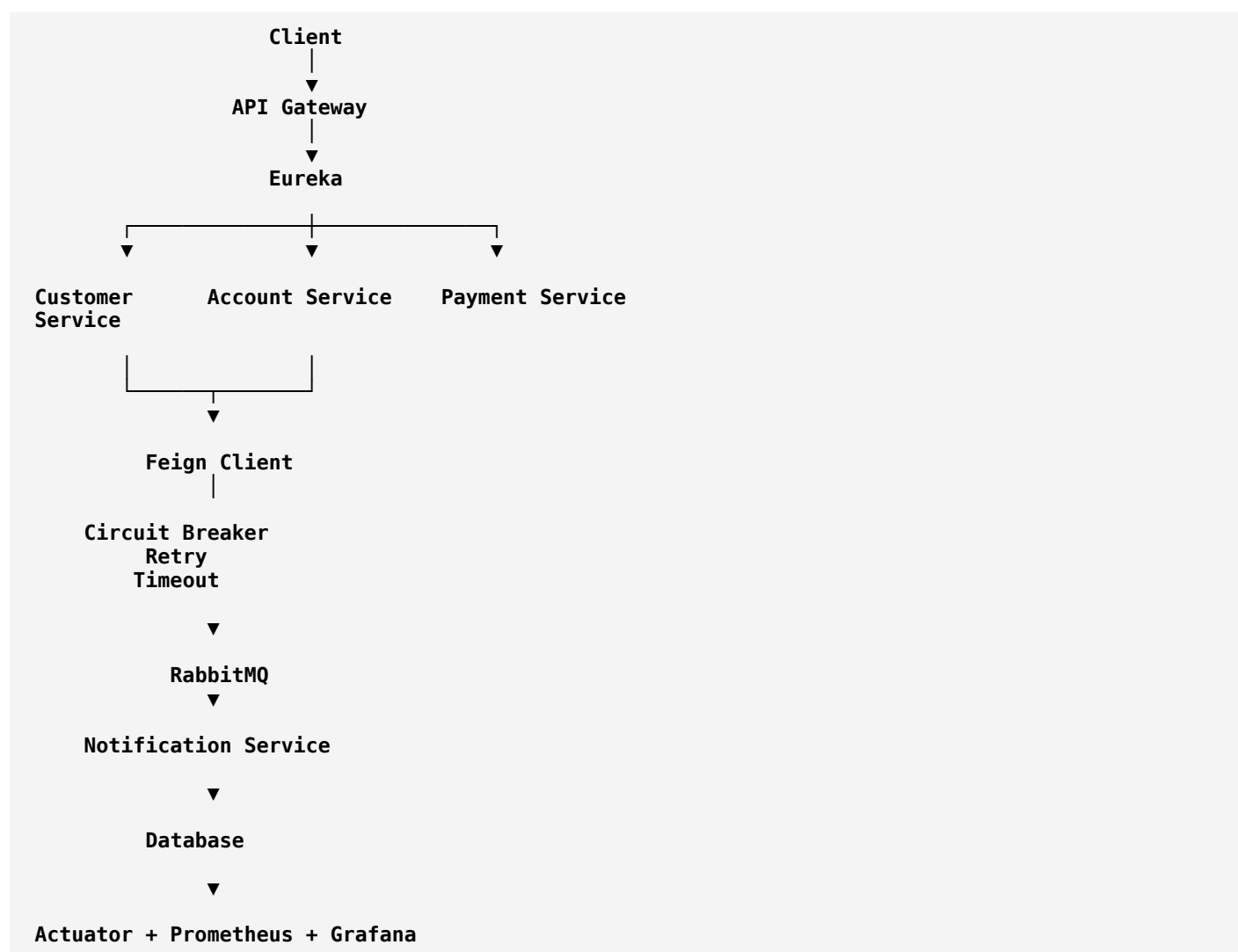
If Step 3 fails:

Compensation Action

Refund Account

Each service performs its own rollback.

Production Architecture



Interview Answer (Short Version)

If one microservice fails:

1. Use **Service Discovery (Eureka)** to locate services.
2. Use **Feign Client** for communication.
3. Configure **timeouts** to avoid hanging requests.
4. Use **Retry** for temporary failures.
5. Use **Circuit Breaker (Resilience4j)** to stop repeated failed calls.
6. Use **Fallback methods** to return default/cached responses.

7. Use **RabbitMQ/Kafka** for asynchronous operations.
8. Use **Saga Pattern** for distributed transaction consistency.
9. Monitor services using **Actuator, Prometheus, and Grafana**.
10. Keep services loosely coupled so one failure doesn't bring down the entire system.

This end-to-end failure handling flow is exactly what many banking and fintech systems use in production.

Explain Circuit Breaker in simple words

Circuit Breaker in Simple Words

Think about an electrical circuit breaker in your house.

When too much current flows:

```
Electric Fault
  ↓
Circuit Breaker Trips
  ↓
Stops Electricity Flow
```

This protects your appliances from damage.

Same Concept in Microservices

Suppose:

```
Payment Service
  ↓
Account Service
```

Payment Service needs to call Account Service.

Normally:

```
Payment Service
  ↓
Account Service
  ↓
Response
```

Everything works fine.

Problem Scenario

Account Service goes down.

```
Payment Service
  ↓
```

Account Service **✗**

Without Circuit Breaker:

```
Request 1 → Fail
Request 2 → Fail
Request 3 → Fail
Request 100 → Fail
```

Payment Service keeps trying.

Problems:

- CPU wasted
- Network wasted
- Threads blocked
- Application becomes slow
- Other services may also fail

This is called a **cascading failure**.

Circuit Breaker Solution

After a certain number of failures:

```
Payment Service
  ↓
Circuit Breaker
  X
Account Service
```

The circuit breaker says:

"Account Service is already down. Don't even try calling it."

Instead, return a fallback response immediately.

Real Example

Imagine you call your friend.

```
Call 1 → Not Reachable
Call 2 → Not Reachable
Call 3 → Not Reachable
```

After that, you stop calling for some time.

That's exactly what a circuit breaker does.

Three States of Circuit Breaker

1. Closed State (Normal)

Everything is working.

```
Payment Service
  ↓
Account Service
```

All requests pass through.

```
Success
Success
Success
```

2. Open State (Service Failed)

Too many failures occurred.

```
Payment Service
  ↓
Circuit Breaker
  X
Account Service
```

No requests are sent.

Immediately returns fallback.

```
Service Temporarily Unavailable
```

This protects the system.

3. Half-Open State (Testing)

After some time:

```
Payment Service
  ↓
Circuit Breaker
  ↓
Account Service
```

Circuit Breaker allows a few requests to test the service.

If Successful

```
Request Success
```

Circuit closes again.

```
Closed State
```

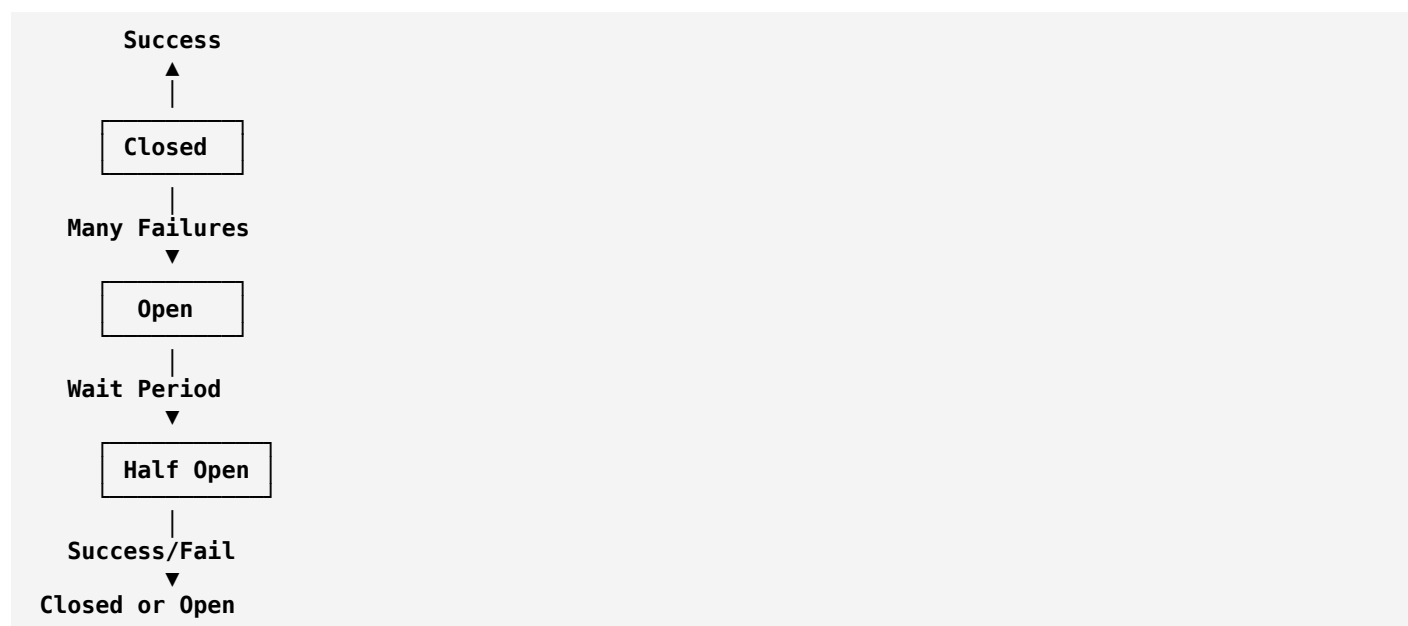

If Failed

Request Failed

Circuit opens again.

Open State

State Flow



Example in Banking

Money Transfer Application

Transfer Service
↓
Account Service

Account Service crashes.

Without Circuit Breaker:

1000 users
1000 failed calls
Server overload

With Circuit Breaker:

Few failures detected
↓
Circuit Opened
↓
No more calls sent
↓
Fallback Response

Users may see:

Account details currently unavailable.
Please try again later.

But the application remains healthy.

Spring Boot Example (Resilience4j)

```
Java
@CircuitBreaker(
    name = "accountService",
    fallbackMethod = "fallbackAccount"
)
public Account getAccount(Long id) {
    return accountClient.getAccount(id);
}
```

Fallback:

```
Java
public Account fallbackAccount(
    Long id,
    Exception ex) {
    return new Account(
        id,
        "Service Unavailable"
    );
}
```

Interview Answer (30 Seconds)

What is Circuit Breaker?

Circuit Breaker is a fault-tolerance pattern used in microservices. When a dependent service starts failing repeatedly, the circuit breaker stops sending requests to that service for a period of time and immediately returns a fallback response. This prevents cascading failures, reduces resource usage, and improves system stability.

Popular implementation in Spring Boot:

- Resilience4j
- (Older projects may use) Netflix Hystrix

Easy Interview One-Liner

Circuit Breaker is like a safety switch that temporarily stops calling a failing service so the rest of the system continues to work normally.

Any thing more i need to know about microservice

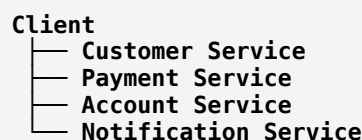
For a Java Spring Boot developer with ~2 years of experience, understanding only Monolith, Microservices, Eureka, Config Server, RabbitMQ, and Circuit Breaker is not enough anymore. Most

interviews expect you to know the complete microservices ecosystem.

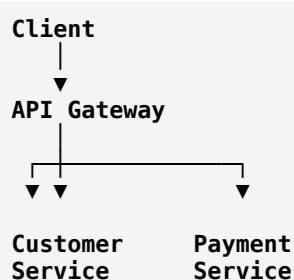
1. API Gateway (Very Important)

Clients should not directly call services.

✗ Bad



✓ Good



Responsibilities:

- Routing
- Authentication
- Authorization
- Rate Limiting
- Logging
- Request Transformation

Popular:

- Spring Cloud Gateway

Interview:

Why API Gateway?

Single entry point for all clients.

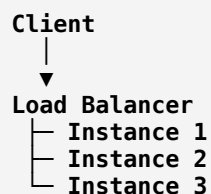
2. Load Balancing

Suppose Payment Service has 3 instances.

```

Payment Service 1
Payment Service 2
Payment Service 3
  
```

Load Balancer distributes requests.



Benefits:

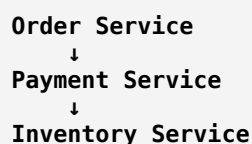
- High Availability
- Scalability

Spring Cloud usually integrates this with service discovery.

3. Distributed Transactions (Saga Pattern)

A common interview favorite.

Suppose:



Payment succeeds.

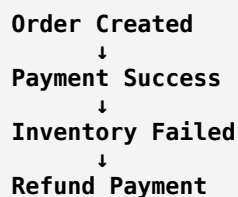
Inventory fails.

What now?

In microservices:

✗ Don't use a single DB transaction across services.

Use Saga Pattern.



Each service performs a compensating action.

Interview:

Why not use @Transactional across microservices?

Because each service has its own database.

4. Database Per Service

Every service owns its database.

```
Customer Service → Customer DB
Payment Service → Payment DB
Order Service → Order DB
```

Benefits:

- Loose coupling
- Independent deployments
- Independent scaling

Interview:

Can two microservices share the same database?

Generally no. It creates tight coupling.

5. Event-Driven Architecture

Instead of:

```
Payment Service
↓
Notification Service
```

Use events.

```
Payment Service
↓
PaymentCompleted Event
↓
RabbitMQ / Kafka
↓
Notification Service
```

Benefits:

- Loose coupling
- Better scalability

6. Kafka vs RabbitMQ

Feature	RabbitMQ	Kafka
Messaging	Queue	Event Streaming
Ordering	Limited	Strong
Throughput	Good	Very High
Replay Messages	No	Yes

Feature	RabbitMQ	Kafka
Banking Notifications	Good	Good
Analytics	Poor	Excellent

Popular interview question:

When would you choose Kafka over RabbitMQ?

For high-volume event streaming and replayable event logs.

7. Distributed Tracing

A user request may travel through:

```

Gateway
↓
Order Service
↓
Payment Service
↓
Notification Service

```

If an error occurs, how do you trace it?

Use Trace IDs.

```

Trace ID:
abc123

```

Every service logs the same ID.

Popular tools:

- Zipkin
- Jaeger

8. Centralized Logging

Without centralized logging:

```

Server A Logs
Server B Logs
Server C Logs

```

Difficult to debug.

Use:

- Elasticsearch
- Logstash
- Kibana

Together called ELK Stack.

9. Monitoring

Need answers to:

- Is service up?
- CPU usage?
- Memory usage?
- Error rate?

Tools:

- Spring Boot Actuator
- Prometheus
- Grafana

10. Security

Very common in interviews.

Usually:

```

Client
  ↓
JWT Token
  ↓
Gateway
  ↓
Microservices
  
```

Authentication:

- JWT
- OAuth2

Interview:

Why JWT in microservices?

Stateless authentication.

11. Service Communication

Two major types:

Synchronous

REST

```
Service A
  ↓
Service B
```

Needs immediate response.

Examples:

- Get customer details
- Check account balance

Asynchronous

RabbitMQ/Kafka

```
Service A
  ↓
Queue
  ↓
Service B
```

Examples:

- Email
- SMS
- Audit logs

Interview:

When would you prefer async communication?

When immediate response is not required.

12. Containerization

Almost every modern microservice runs inside containers.

Popular:

Docker

```
Customer Service Container
Payment Service Container
Notification Service Container
```

Benefits:

- Same behavior everywhere
- Easy deployment

13. Orchestration

Managing hundreds of containers manually is impossible.

Use:

Kubernetes

Responsibilities:

- Auto-scaling
- Self-healing
- Load balancing
- Rolling deployments

Interview:

What happens if a pod crashes?

Kubernetes automatically creates a new one.

14. CAP Theorem

Advanced but frequently asked.

A distributed system can provide only two of:

Consistency
Availability
Partition Tolerance

Microservices usually favor:

Availability
+
Partition Tolerance

and accept eventual consistency.

15. Eventual Consistency

In monolith:

Everything updated immediately

In microservices:

Payment Success
↓
Notification Updated Later

Data may not be instantly consistent.

Eventually it becomes consistent.

Topics You Should Definitely Know for Interviews

Priority order:

1. Monolith vs Microservices
2. API Gateway
3. Eureka Service Discovery
4. Config Server
5. Feign Client
6. REST Communication
7. RabbitMQ/Kafka
8. Circuit Breaker (Resilience4j)
9. Retry and Fallback
10. Saga Pattern
11. JWT Security
12. Distributed Tracing
13. Docker
14. Kubernetes
15. Monitoring (Actuator, Prometheus, Grafana)

If you're preparing for Spring Boot developer interviews in product companies or fintech/banking organizations, mastering these 15 topics will cover the majority of microservices-related questions asked at the 2–5 years experience level.

Saga Design Pattern for transaction in different microservices
Gateway design pattern CORS here entry point for all microservice

These two topics are asked very frequently in microservice interviews.

1. Saga Pattern

Why Saga Pattern is Needed?

In a monolithic application:

Java

```
@Transactional
public void placeOrder() {
    createOrder();
    makePayment();
    updateInventory();
}
```

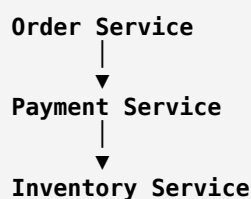
If any step fails:

Rollback Everything

because there is one database transaction.

Problem in Microservices

Suppose:



Each service has its own database.

```
Order DB
Payment DB
Inventory DB
```

Now suppose:

Step 1

Order created 

Step 2

Payment successful 

Step 3

Inventory update failed 

How do you rollback Payment and Order?

You cannot use:

```
Java
@Transactional
```

across multiple databases/services.

Solution: Saga Pattern

A Saga is a sequence of local transactions.

Each service:

1. Performs its own transaction.
 2. If failure occurs, executes a compensating transaction (undo operation).
-

Example

Online Shopping

Step 1

Order Service

Create Order

Status:

PENDING

Step 2

Payment Service

Debit Amount

Success

Step 3

Inventory Service

Reduce Stock

Failed

Now Saga triggers compensation.

Compensation

Inventory Failed

↓

Refund Payment

↓

Cancel Order

Final Result

Order Cancelled
Payment Refunded
Inventory Unchanged

System remains consistent.

Saga Flow

```

graph TD
    OS[Order Service] --> CO[Create Order]
    CO --> PS[Payment Service]
    PS --> DM[Debit Money]
    DM --> IS[Inventory Service]
    IS --> SUFF[Stock Update Failed]
    SUFF --> PC[Payment Compensation]
    PC --> RM[Refund Money]
    RM --> OC[Order Compensation]
    OC --> C[Cancel Order]
  
```

Types of Saga

1. Choreography Saga

Services communicate using events.

```

graph TD
    OCE[Order Created Event] --> PS[Payment Service]
    PS --> PSE[Payment Success Event]
    PSE --> IS[Inventory Service]
  
```

No central coordinator.

Usually implemented using:

- RabbitMQ
- Apache Kafka

Advantages

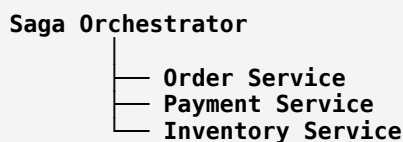
- Simple
- Decentralized

Disadvantages

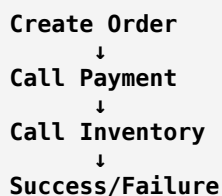
- Difficult to track
- Complex debugging

2. Orchestration Saga

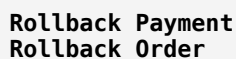
A central service controls everything.



Flow:



If failure:



Advantages

- Easy monitoring
- Better control

Disadvantages

- Extra orchestrator service

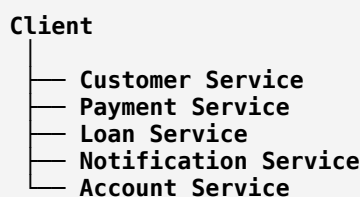
Interview Answer

What is Saga Pattern?

Saga Pattern is a distributed transaction pattern used in microservices. Since each microservice has its own database and we cannot use a single database transaction across services, Saga executes a sequence of local transactions and uses compensating transactions to rollback previous steps if a later step fails.

2. API Gateway Pattern

Problem Without Gateway

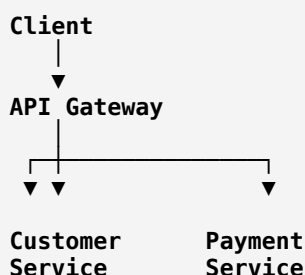


Problems:

- Too many endpoints
- Authentication repeated everywhere
- Difficult monitoring
- Tight coupling

Solution: API Gateway

Single entry point.



All requests first go to Gateway.

Responsibilities of Gateway

1. Routing

Example:

```
/api/customers/**
```

↓

Customer Service

```
/api/payments/**
```

↓

Payment Service

Spring Gateway config:

```
YAML
routes:
  - id: customer-service
    uri: lb://customer-service
    predicates:
      - Path=/customers/**
```

2. Authentication

Instead of validating JWT in every service:

```
Client
|
JWT Token
▼
Gateway
```

Gateway validates token once.

Only valid requests move ahead.

3. Authorization

Example:

```
Admin
Can access Loan Service

Customer
Cannot access Loan Service
```

Gateway checks permissions.

4. Logging

All requests pass through Gateway.

```
Who called?
When?
Response Time?
```

Easy monitoring.

5. Rate Limiting

Prevent abuse.

Example:

```
Maximum 100 Requests/Minute
```

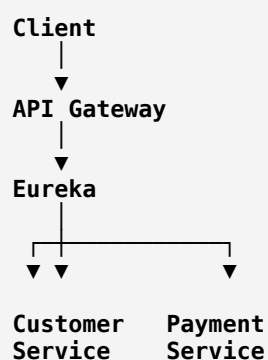
If exceeded:

```
http
429 Too Many Requests
```

Gateway + Eureka

Usually used together.

Architecture:



Gateway asks Eureka:

```
Where is Payment Service?
```

Eureka returns:

```
10.0.0.15:8083
```

Gateway forwards request.

Where Does CORS Come In?

Suppose:

Frontend:

```
http://localhost:4200
```

Angular

Backend:

```
http://localhost:8080
```

Gateway

Different origins.

Browser blocks request.

Error:

```
Access-Control-Allow-Origin  
Missing
```

This is a CORS issue.

CORS in Microservices

Instead of configuring CORS in every service:

```
Customer Service  
Payment Service  
Loan Service  
Notification Service
```

Configure it once in Gateway.

Example:

```
Java
@Bean
public CorsWebFilter corsFilter() {
    CorsConfiguration config =
        new CorsConfiguration();

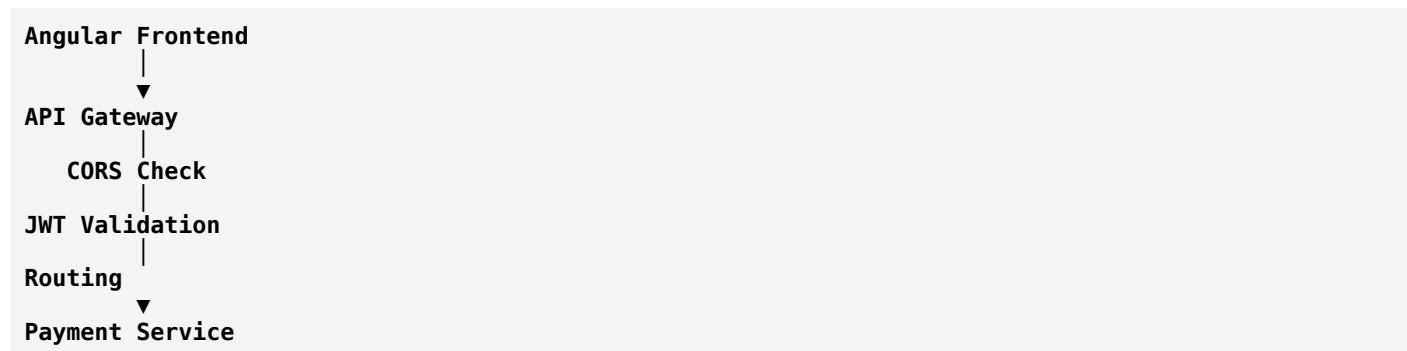
    config.addAllowedOrigin("*");
    config.addAllowedMethod("*");
    config.addAllowedHeader("*");

    UrlBasedCorsConfigurationSource source =
        new UrlBasedCorsConfigurationSource();

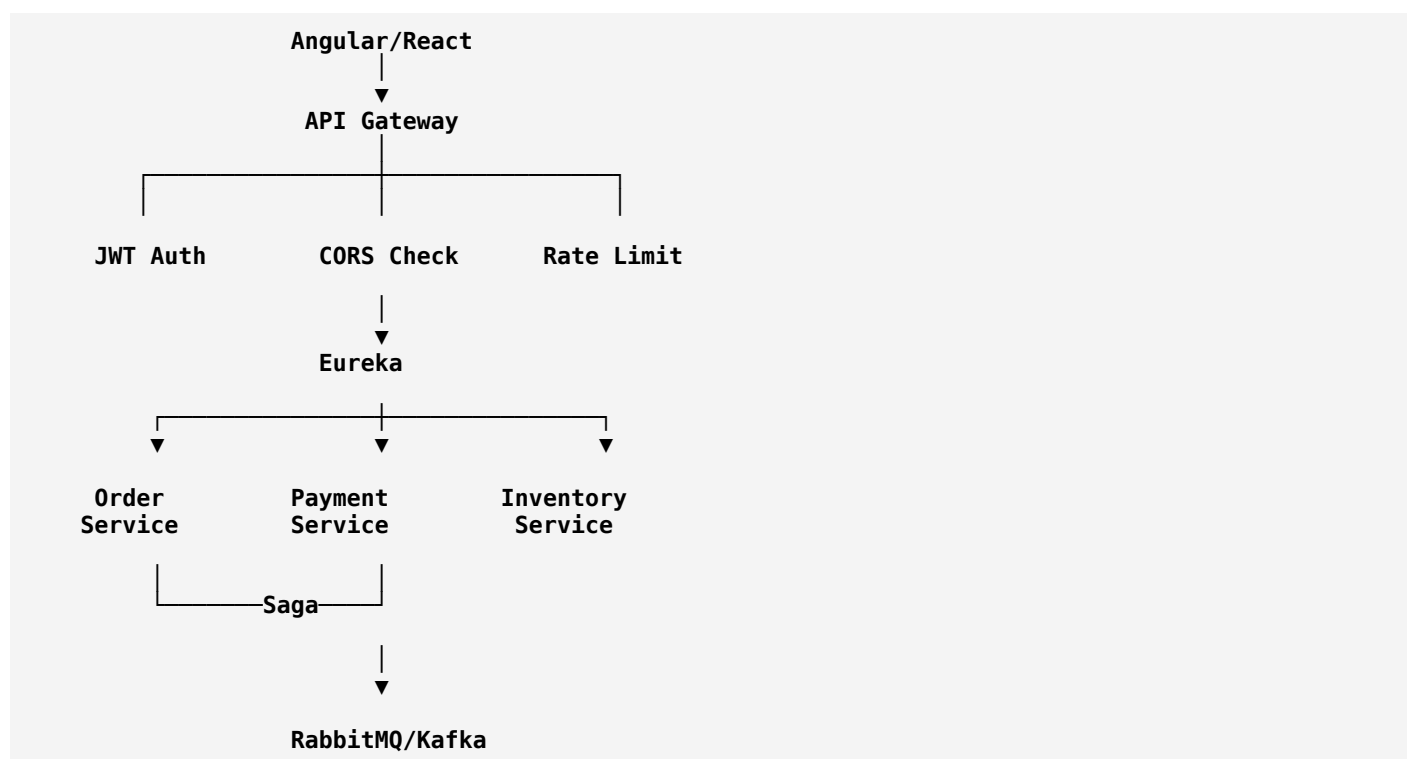
    source.registerCorsConfiguration(
        "/*",
```

```
config);  
  
return new CorsWebFilter(source);  
}
```

Request Flow with Gateway + CORS



Complete Production Architecture



Interview One-Liners

Saga Pattern

Used to manage distributed transactions in microservices through local transactions and compensating actions.

API Gateway

A single entry point for all client requests that handles routing, authentication, authorization, logging, rate limiting, and CORS before forwarding requests to microservices.

Why configure CORS at Gateway?

Because every client request enters through the Gateway, so CORS can be managed centrally instead of configuring it in each microservice separately.

Explain Kafka everything and definitions how it works
Interview Questions asked related to kafka and scenarios

What is Offset in Kafka?

An **Offset** is a unique sequence number assigned to each message within a Kafka partition.

Think of it like a page number in a book.

Book **Pages**

Page 1
Page 2
Page 3
Page 4

Similarly, in Kafka:

Partition-0

Offset 0 → **Payment Completed**
Offset 1 → **Payment Completed**
Offset 2 → **Payment Completed**
Offset 3 → **Payment Completed**

Every new message gets the next offset number.

Why is Offset Needed?

Kafka needs to know:

- Which messages have been consumed
- Which messages are pending
- From where a consumer should restart

Without offsets, Kafka wouldn't know where the consumer left off.

Example

Suppose Payment Service publishes 4 events.

Topic: **payment-events**

Partition-0

Offset 0 → **Payment #1001**
Offset 1 → **Payment #1002**
Offset 2 → **Payment #1003**
Offset 3 → **Payment #1004**

Notification Service starts consuming.

After processing first two messages:

```
Consumed:  
Offset 0  
Offset 1
```

Current Consumer Offset:

```
2
```

Meaning:

"Next time, start reading from Offset 2."

Consumer Crash Scenario

Suppose consumer processes:

```
Offset 0 ✓  
Offset 1 ✓  
Offset 2 ✓
```

Then crashes.

Kafka has stored:

```
Last Committed Offset = 3
```

After restart:

```
Start Reading From Offset 3
```

No need to read old messages again.

Offset is Partition Specific

Offsets are unique only within a partition.

Example:

```
Topic: payment-events  
  
Partition-0  
Offset 0  
Offset 1  
Offset 2  
  
Partition-1  
Offset 0  
Offset 1  
Offset 2
```

Notice both partitions have their own offset sequence.

Consumer Group Example

Topic

Partition-0
Partition-1
Partition-2

Consumer Group:

Consumer-1 → Partition-0

Consumer-2 → Partition-1

Consumer-3 → Partition-2

Each consumer tracks offsets independently.

Example:

Consumer-1 Current Offset = 50

Consumer-2 Current Offset = 100

Consumer-3 Current Offset = 75

Current Offset vs End Offset

Suppose:

Partition End Offset = 1000
Consumer Offset = 950

Lag:

$1000 - 950 = 50$

Consumer is behind by 50 messages.

This is called **Consumer Lag**.

Interviewers often ask:

What is Consumer Lag?

Answer:

The difference between the latest offset in a partition and the last consumed offset.

Offset Commit

After processing a message, consumer can commit its offset.

Auto Commit

Kafka automatically saves offsets.

```
properties
enable.auto.commit=true
```

Manual Commit

Developer controls when offset is saved.

```
Java
ack.acknowledge();
```

Used when you want better reliability.

Real Banking Example

```
Payment Service
  ↓
Kafka Topic
  ↓
Notification Service
```

Messages:

```
Offset 0 → Transaction 1001
Offset 1 → Transaction 1002
Offset 2 → Transaction 1003
```

Notification Service processes:

```
1001 ✓
1002 ✓
```

and commits:

```
Offset = 2
```

If service crashes:

After restart:

```
Start From Offset 2
```

Only Transaction 1003 is processed.

Most Asked Interview Question

Does Kafka delete a message after it is consumed?

No.

Unlike RabbitMQ, Kafka keeps messages for a configured retention period.

Example:

```
Retention = 7 Days
```

Even if Offset 100 is consumed, Kafka still stores it for 7 days.

That's why Kafka supports **message replay**.

Easy Interview Definition

Offset is the unique position of a message within a Kafka partition. Kafka uses offsets to track which messages have been consumed and from where a consumer should continue reading after a restart.

Quick Example

```
Partition-0
```

```
Offset 0 → Message A
Offset 1 → Message B
Offset 2 → Message C
Offset 3 → Message D
```

If the consumer has committed offset **2**, it means:

```
A and B processed
Next message to read = C (Offset 2)
```

This simple concept is the foundation of Kafka's reliability, replay capability, and consumer recovery.

can multiple consumer consume same partition

Excellent interview question.

Short Answer

Yes and No, depending on the Consumer Group.

Scenario 1: Same Consumer Group

Suppose:

```
Topic: payment-events
Partition-0
```

Consumer Group:


```
notification-group
```

```
Consumer-1  
Consumer-2
```

Can both consumers read Partition-0 simultaneously?

Answer: NO

Kafka guarantees:

One partition can be assigned to only one consumer within the same consumer group.

Example:

```
Partition-0 → Consumer-1
```

```
Consumer-2 → No partition assigned
```

This prevents duplicate processing.

Why?

Imagine banking transactions:

```
Transaction #1001
```

If both consumers process it:

```
Consumer-1 → Send SMS
```

```
Consumer-2 → Send SMS
```

Customer receives duplicate notifications.

Kafka avoids this by assigning a partition to only one consumer in a group.

Scenario 2: Different Consumer Groups

Now:

```
Topic: payment-events
```

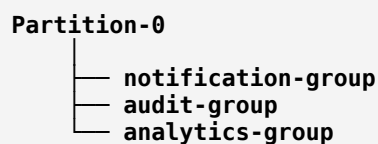
Consumer Groups:

```
notification-group
```

```
audit-group
```

```
analytics-group
```

Each group can consume the same partition.



All three get the same message.

Example

Payment Event:

```

JSON
{
  "transactionId":1001,
  "amount":5000
}
  
```

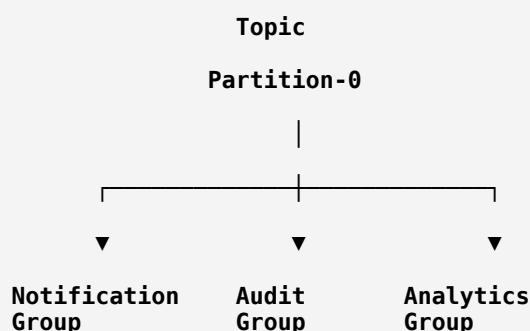
Consumed by:

```

notification-group
  ↓
Send SMS
audit-group
  ↓
Store Audit Log
analytics-group
  ↓
Generate Reports
  
```

Same message, different business purposes.

Interview Diagram



Each group maintains its own offset.

What Happens If More Consumers Than Partitions?

Example:

Topic

Partition-0
Partition-1

Consumers:

Consumer-1
Consumer-2
Consumer-3

Assignment:

Partition-0 → Consumer-1
Partition-1 → Consumer-2
Consumer-3 → Idle

One consumer gets no work.

What Happens If More Partitions Than Consumers?

Example:

Partition-0
Partition-1
Partition-2
Partition-3

Consumers:

Consumer-1
Consumer-2

Assignment:

Consumer-1
├─ Partition-0
└─ Partition-2

Consumer-2
├─ Partition-1
└─ Partition-3

One consumer can handle multiple partitions.

Most Asked Interview Question

Can multiple consumers consume the same partition?

Answer:

- In the **same consumer group** →  No, only one consumer gets a partition.
- In **different consumer groups** →  Yes, each group can read the same partition independently.

Follow-up Interview Question

Why doesn't Kafka allow multiple consumers from the same group to consume the same partition?

Because Kafka guarantees:

- Message ordering within a partition
- No duplicate processing within a consumer group

If two consumers processed the same partition simultaneously:

```
Message-1
Message-2
Message-3
```

Order could break and duplicates could occur.

Real Banking Example

```
Topic: transaction-events
```

```
Partition-0
```

Groups:

```
notification-group
fraud-group
audit-group
```

A single transaction event can be consumed by:

- SMS/Email Notification Service
- Fraud Detection Service
- Audit Logging Service

All at the same time because they belong to different consumer groups.

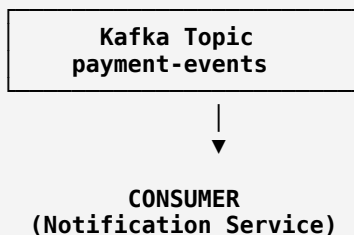
This is one of Kafka's biggest strengths: **publish once, consume many times by different services.**

give diagram of kafka which includes all scenarios

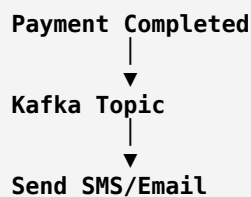
Here are the most important Kafka diagrams that cover almost every interview scenario.

1. Basic Kafka Architecture

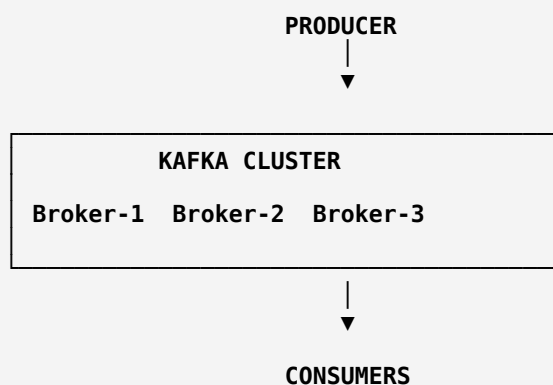
```
PRODUCER
(Payment Service)
↓
```



Example:



2. Kafka Cluster Architecture



Broker = Kafka Server

Cluster = Collection of Brokers

3. Topic, Partition and Offset

```

Topic : payment-events
Partition-0
Offset 0 -> Payment #1001
Offset 1 -> Payment #1002
Offset 2 -> Payment #1003
Offset 3 -> Payment #1004
Offset 4 -> Payment #1005
  
```

Offset = Position of message.

4. Multiple Partitions

Topic : payment-events

Partition-0
Offset 0
Offset 1
Offset 2

Partition-1
Offset 0
Offset 1
Offset 2

Partition-2
Offset 0
Offset 1
Offset 2

Offsets are unique only inside a partition.

5. Consumer Group Architecture

Topic : payment-events

Partition-0
Partition-1
Partition-2



Consumer Group

Consumer-1
Consumer-2
Consumer-3

Assignment:

Partition-0 → Consumer-1

Partition-1 → Consumer-2

Partition-2 → Consumer-3

6. More Consumers than Partitions

Topic

Partition-0
Partition-1

Consumers

Consumer-1

Consumer-2
Consumer-3

Assignment:

Partition-0 → Consumer-1
Partition-1 → Consumer-2
Consumer-3 → Idle

Interview Question:

Can Kafka utilize all consumers?

Answer:

No. Number of active consumers $\leq$ Number of partitions.

7. More Partitions than Consumers

Partitions

P0
P1
P2
P3

Consumers

Consumer-1
Consumer-2

Assignment:

Consumer-1
└─ P0
└─ P2

Consumer-2
└─ P1
└─ P3

One consumer can read multiple partitions.

8. Same Consumer Group Scenario

Consumer Group A

Consumer-1
Consumer-2

Topic

Partition-0

Assignment:

Partition-0 → Consumer-1

Consumer-2 → No Access

Important Rule:

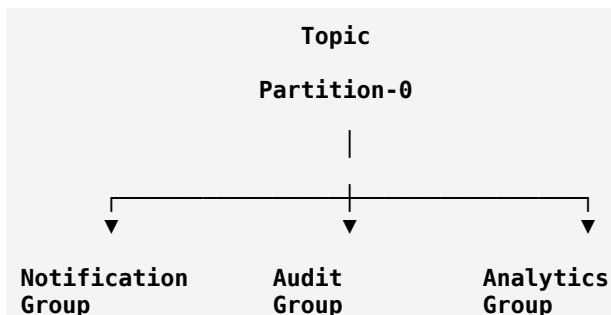
One Partition

=

One Consumer

within same group

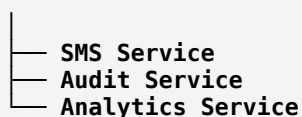
9. Different Consumer Groups



All groups receive same message.

Example:

Payment Event



This is why Kafka is excellent for Event Driven Architecture.

10. Replication Architecture

Topic : payment-events

Leader

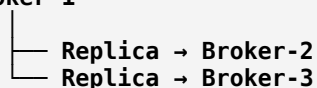
Broker-1

Followers

Broker-2

Broker-3

Broker-1



11. Broker Failure Scenario

Before Failure


```
Broker-1 (Leader)
Broker-2 (Follower)
Broker-3 (Follower)
```

After Failure

```
Broker-1 ✖
Broker-2 (New Leader)
Broker-3 (Follower)
```

Kafka automatically elects a new leader.

No downtime.

12. Offset Commit Scenario

```
Partition-0
```

```
Offset 0
Offset 1
Offset 2
Offset 3
Offset 4
```

Consumer processed:

```
Offset 0 ✓
Offset 1 ✓
Offset 2 ✓
```

Committed Offset:

```
3
```

Consumer crashes.

Restart:

```
Start reading from Offset 3
```

No message loss.

13. Consumer Lag Scenario

```
Partition End Offset = 1000
Consumer Offset = 850
```

Lag:

1000 - 850

Lag = 150

Meaning:

150 Messages Pending

14. Rebalancing Scenario

Before

Consumer-1 → P0

Consumer-2 → P1

New Consumer Joins

Consumer-1

Consumer-2

Consumer-3

Kafka Rebalances

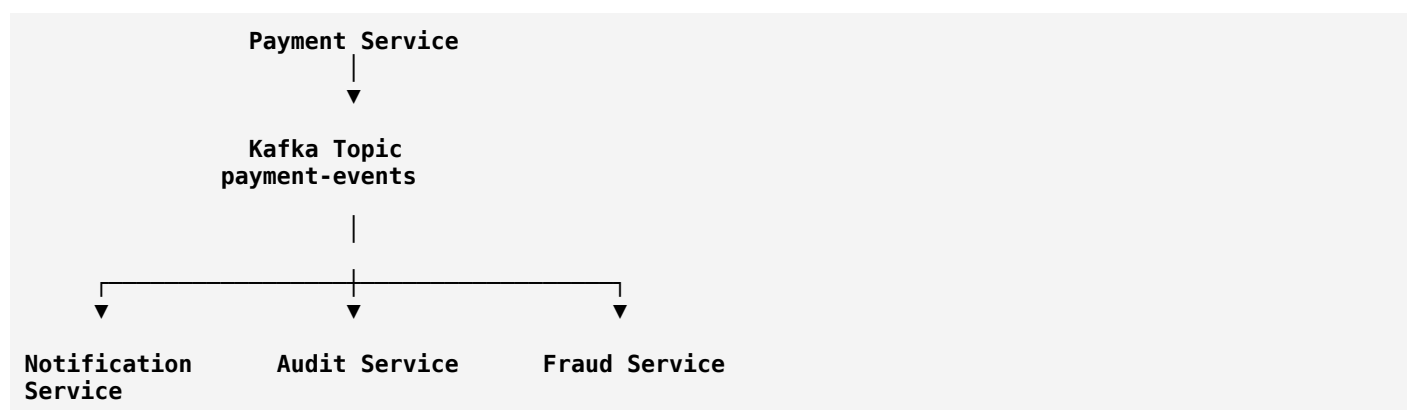
Consumer-1 → P0

Consumer-2 → P1

Consumer-3 → P2

Partitions redistributed automatically.

15. Event Driven Microservice Architecture

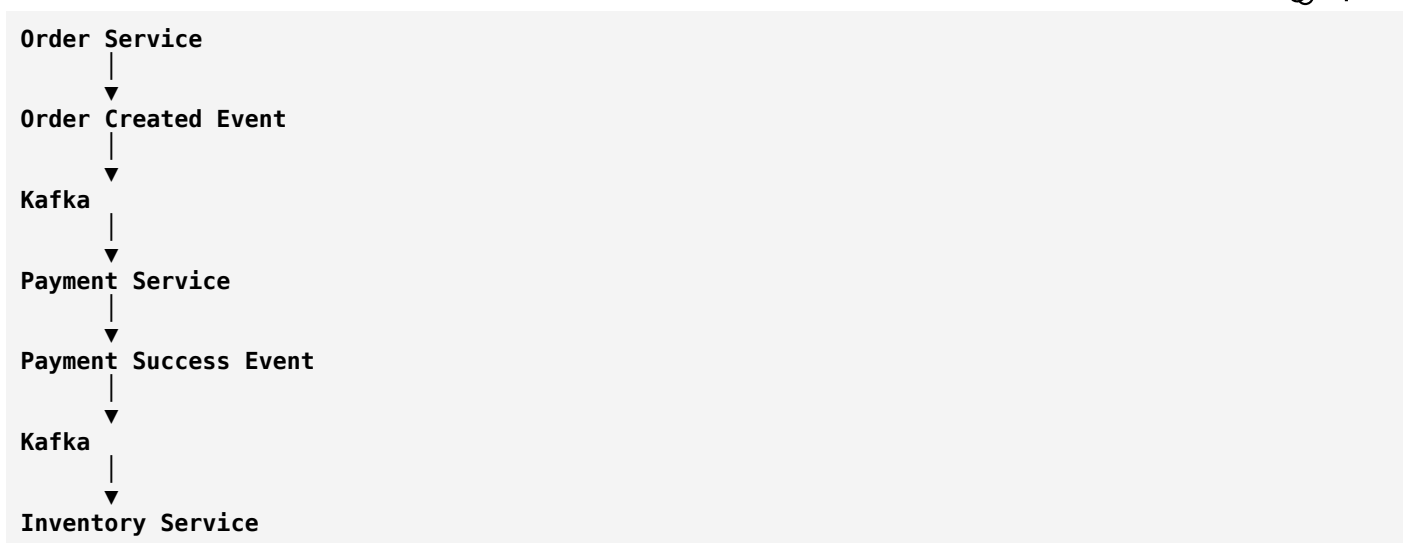


One event.

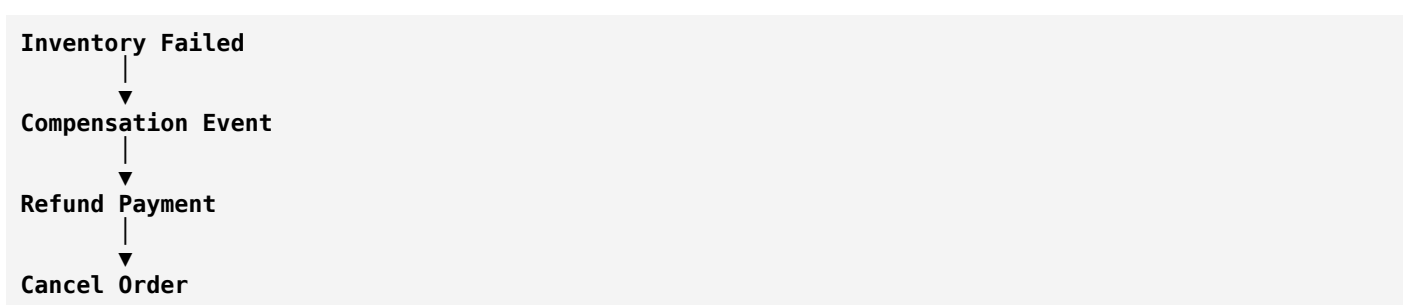
Multiple consumers.

No direct service-to-service dependency.

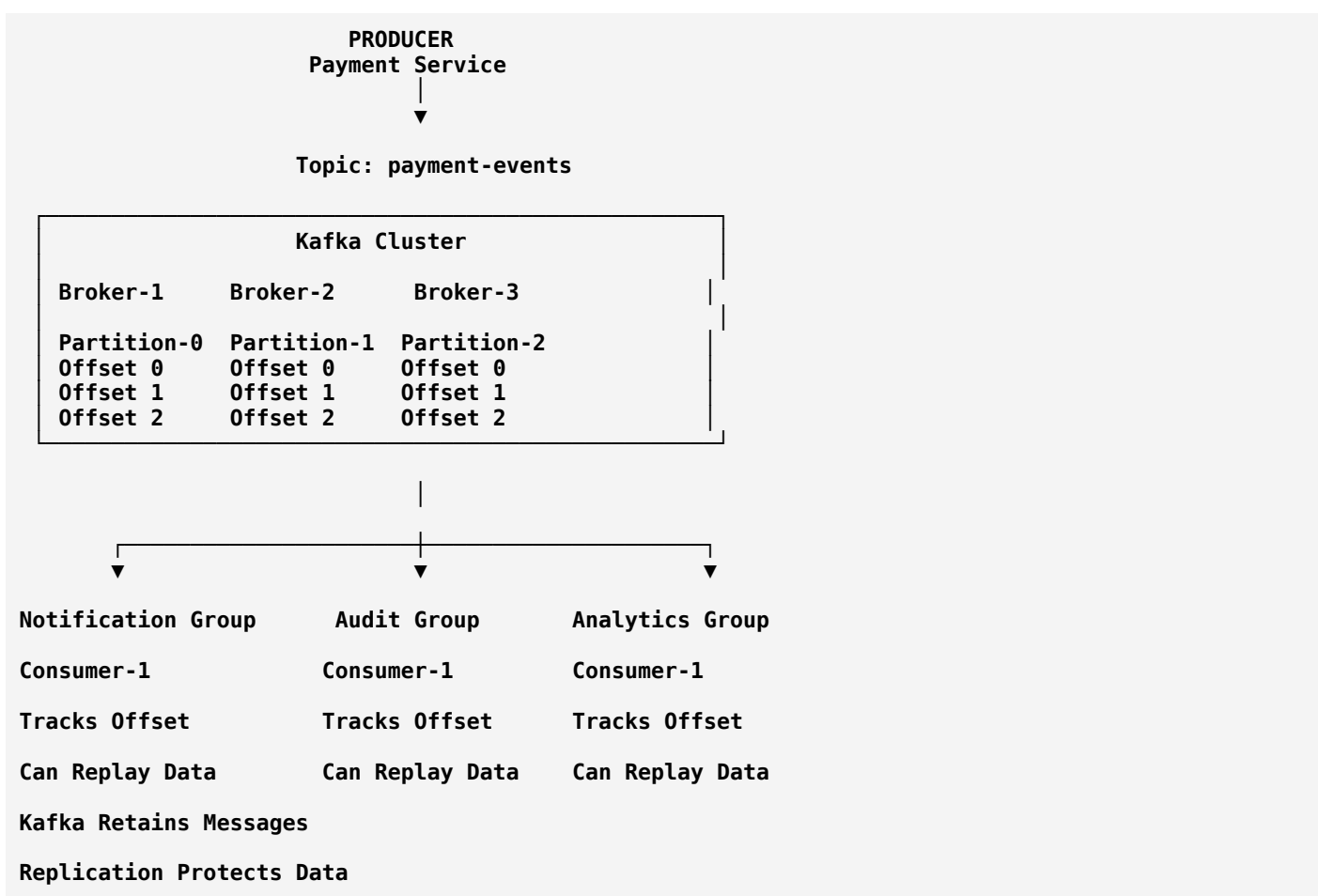
16. Kafka + Saga Pattern



Failure:



Complete Kafka Interview Diagram



Rebalancing Handles Consumer Failures

Offsets Enable Recovery

Consumer Groups Enable Scalability

If you can draw and explain these 16 diagrams in an interview, you can confidently answer around **90% of Kafka architecture and scenario-based questions** asked for Spring Boot/Microservices roles.

SOLID Principles in Java explain with examples

SOLID Principles in Java

SOLID is a set of 5 design principles that help write:

- Maintainable code
- Reusable code
- Scalable code
- Testable code

Created by Robert C. Martin.

S = Single Responsibility Principle (SRP)

Definition

A class should have only one reason to change.

A class should have only one responsibility.

Bad Example

```
Java
class Employee {
    public void calculateSalary() {
        // salary logic
    }

    public void saveToDatabase() {
        // database logic
    }

    public void generateReport() {
        // report logic
    }
}
```

Problems:

Employee Class

Salary Logic
Database Logic
Report Logic

Multiple responsibilities.

Good Example

Java

```
class Employee {
    private String name;
}
```

Java

```
class SalaryService {
    public void calculateSalary() {
    }
}
```

Java

```
class EmployeeRepository {
    public void save() {
    }
}
```

Java

```
class ReportService {
    public void generateReport() {
    }
}
```

Each class has one responsibility.

Interview Question

Why SRP?

Because changes in one functionality should not affect unrelated functionality.

O = Open Closed Principle (OCP)

Definition

Software entities should be open for extension but closed for modification.

Add new functionality without modifying existing code.

Bad Example

```
Java
class PaymentService {
    public void pay(String type) {
        if(type.equals("CARD")) {
        } else if(type.equals("UPI")) {
        }
    }
}
```

Every new payment method requires code modification.

Good Example

```
Java
interface Payment {
    void pay();
}
```

```
Java
class CardPayment implements Payment {
    public void pay() {
        System.out.println("Card Payment");
    }
}
```

```
Java
class UpiPayment implements Payment {
    public void pay() {
        System.out.println("UPI Payment");
    }
}
```

```
Java
class NetBankingPayment implements Payment {
    public void pay() {
        System.out.println("Net Banking");
    }
}
```

Usage:

```
Java
```

```
Payment payment = new UpiPayment();
payment.pay();
```

New payment type?

Create new class only.

Existing code remains unchanged.

Interview Answer

OCP promotes extensibility without changing tested code.

L = Liskov Substitution Principle (LSP)

Definition

Child class should be able to replace parent class without breaking behavior.

Bad Example

```
Java
class Bird {
    public void fly() {
    }
}
```

```
Java
class Penguin extends Bird {
    public void fly() {
        throw new RuntimeException();
    }
}
```

Problem:

```
Java
Bird bird = new Penguin();
bird.fly();
```

Crash.

Penguin cannot truly replace Bird.

LSP violated.

Good Example

```
Java
interface Bird {
}
```

```
Java
interface FlyingBird {
    void fly();
}
```

```
Java
class Sparrow implements Bird, FlyingBird {
    public void fly() {
    }
}
```

```
Java
class Penguin implements Bird {
}
```

Now behavior is correct.

Interview Question

Real-world LSP Example?

Square and Rectangle inheritance is a famous LSP violation.

I = Interface Segregation Principle (ISP)

Definition

Clients should not be forced to implement methods they don't use.

Bad Example

```
Java
interface Worker {
    void work();
    void eat();
    void sleep();
}
```


Robot:

```
Java
class Robot implements Worker {
    public void work() {
    }

    public void eat() {
        throw new RuntimeException();
    }

    public void sleep() {
        throw new RuntimeException();
    }
}
```

Bad design.

Good Example

```
Java
interface Workable {
    void work();
}
```

```
Java
interface Eatable {
    void eat();
}
```

```
Java
interface Sleepable {
    void sleep();
}
```

Human:

```
Java
class Human implements
    Workable,
    Eatable,
    Sleepable {
}
```

Robot:

```
Java
class Robot implements Workable {
}
```

Perfect.

Interview Question

Why ISP?

Avoid unnecessary implementation and tight coupling.

D = Dependency Inversion Principle (DIP)

Definition

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Bad Example

```
Java
class MySQLDatabase {
    public void save() {
    }
}
```

```
Java
class UserService {
    private MySQLDatabase db =
        new MySQLDatabase();
}
```

Problem:

Switching to Oracle requires changing UserService.

Tight coupling.

Good Example

```
Java
interface Database {
    void save();
}
```

MySQL:

```
Java
class MySQLDatabase
    implements Database {
```

```
public void save() {
}
```

Oracle:

```
Java
class OracleDatabase
    implements Database {

    public void save() {
    }
}
```

Service:

```
Java
class UserService {

    private Database database;

    UserService(Database database) {

        this.database = database;
    }
}
```

Usage:

```
Java
Database db =
    new MySQLDatabase();

UserService service =
    new UserService(db);
```

Now UserService depends on abstraction.

How Spring Uses DIP

Spring Dependency Injection:

```
Java
@Service
class UserService {

    @Autowired
    UserRepository repository;
}
```

Spring injects implementation.

This is DIP in action.

Easy Banking Example

Without SOLID

```
PaymentService
```

```
Card Payment
UPI Payment
Net Banking
Database
Report
Notification
```

Huge class.

Difficult to maintain.

With SOLID

```
Payment Interface
```

```
CardPayment
UPIPayment
NetBankingPayment
```

```
NotificationService
```

```
PaymentRepository
```

```
ReportService
```

Clean architecture.

Most Asked Interview Questions

What is SOLID?

Five object-oriented design principles that improve maintainability and scalability.

Which SOLID principle is most used in Spring Boot?

Dependency Inversion Principle (DIP)

because of:

```
Java
```

```
@Autowired
Constructor Injection
Bean Injection
```

Which design pattern follows OCP?

- Strategy Pattern
- Factory Pattern

Which SOLID principle is violated by large God Classes?

Which principle is achieved using interfaces?

- OCP
- ISP
- DIP

Quick Revision Table

Principle	Meaning
S	Single Responsibility Principle
O	Open Closed Principle
L	Liskov Substitution Principle
I	Interface Segregation Principle
D	Dependency Inversion Principle

One-Line Definitions

- **SRP** → One class, one responsibility.
- **OCP** → Extend behavior without modifying existing code.
- **LSP** → Child should replace parent without breaking functionality.
- **ISP** → Don't force classes to implement unused methods.
- **DIP** → Depend on abstractions, not concrete implementations.

For Spring Boot interviews, be ready to explain **SOLID + Design Patterns (Singleton, Factory, Strategy, Builder, Observer)** together, because interviewers often ask how SOLID principles are applied through design patterns.

Explain Redis in Microservice with Example Advantages
How Redis Works
Redis Configuration
How to set Data in cache?
Interview Questions and Senarios for Redis

Redis in Microservices - Complete Interview Guide

Redis is an in-memory database commonly used for:

- Caching
- Session Management

- Rate Limiting
- Distributed Locking
- Pub/Sub Messaging

In Spring Boot microservices, Redis is mostly used as a **cache**.

Why Redis is Needed?

Suppose:



Every request hits the database.

Example:

```

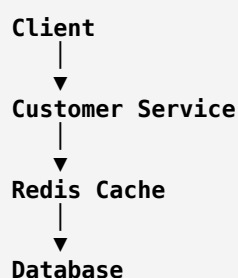
SQL
SELECT * FROM customer WHERE id=101;
  
```

If 10,000 users request the same customer:

10000 DB Queries

Database becomes slow.

Solution: Redis Cache



Flow:

1. Check Redis
2. If data found → return
3. If not found → query DB
4. Store result in Redis
5. Return response

Real Banking Example

Customer profile:

```
JSON
{
  "customerId": 101,
  "name": "John"
}
```

Stored in Redis.

Next request:

No DB Hit

Data comes directly from memory.

Why Redis is Fast?

Database:

Disk

Redis:

RAM (Memory)

RAM is much faster than disk.

Typical response:

Database → 10-100 ms

Redis → <1 ms

How Redis Works?

Imagine Redis as a giant HashMap in memory.

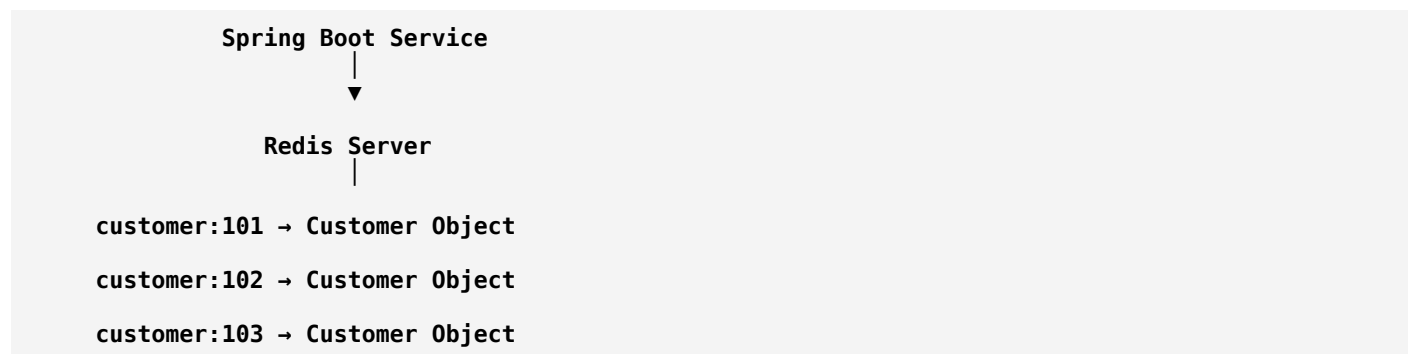
Key	Value
customer:101	-> Customer Object
customer:102	-> Customer Object
account:5001	-> Account Object

Lookup:

Get customer:101

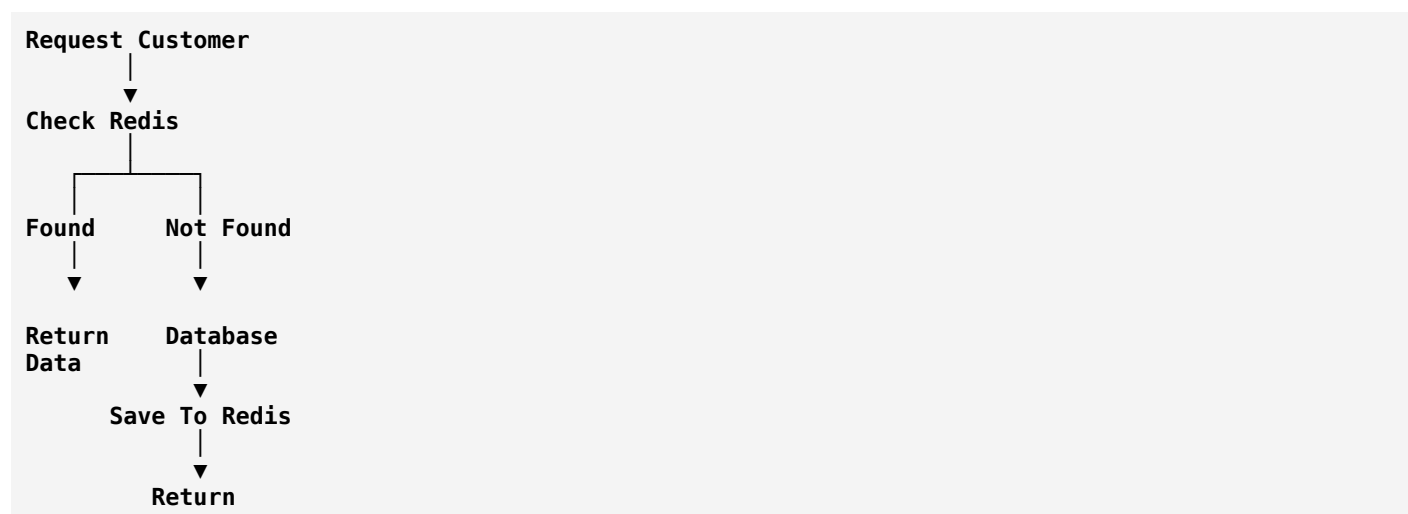
Very fast.

Redis Architecture



Cache Flow (Cache Aside Pattern)

Most common interview topic.



Spring Boot Redis Configuration

Dependencies

```

XML
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-data-redis
  </artifactId>
</dependency>
  
```

application.yml

```

YAML
  
```



```
spring:
  redis:
    host: localhost
    port: 6379
```

Default Redis port:

6379

Enable Cache

```
Java
@SpringBootApplication
@EnableCaching
public class App {
}
```

How to Set Data in Cache?

Using Spring Cache.

Read Data

```
Java
@Cacheable(
  value = "customers",
  key = "#id"
)
public Customer getCustomer(
  Long id) {

  return repository.findById(id)
    .orElseThrow();
}
```

First Request

```
Redis Miss
  |
  v
Database
  |
  v
Save In Redis
```

Second Request

```
Redis Hit
  |
  v
Return Data
```

No database call.

Updating Cache

```
Java
@CachePut(
    value = "customers",
    key = "#customer.id"
)
public Customer updateCustomer(
    Customer customer){

    return repository.save(customer);
}
```

Updates DB and cache.

Remove Cache

```
Java
@CacheEvict(
    value = "customers",
    key = "#id"
)
public void deleteCustomer(
    Long id){
}
```

Removes cached value.

Redis Data Structures

String

```
customer:101
```

Most common.

Hash

```
customer
id=101
name=John
```

Like Java Map.

List

Notifications

Ordered collection.

Set

Unique Values

No duplicates.

Sorted Set

Leaderboard

Ranking system.

Redis TTL (Time To Live)

Cache should not live forever.

Example:

Customer Cache

TTL = 300 Seconds

After 5 minutes:

Automatically Deleted

Example:

Java

```
@Cacheable("customers")
```

Redis:

```
customer:101
```

```
TTL = 300 sec
```

Redis in Microservices

Suppose:

Customer Service

Account Service

Loan Service

Each service may have its own cache.

Customer Service

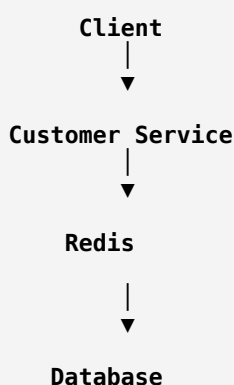
↓
Redis

Account Service

↓
Redis

Or a shared Redis cluster.

Distributed Cache Architecture



Session Management

Without Redis:

User Login

Server-1

User request goes to:

Server-2

Session lost.

Using Redis:

Server-1

↓
Redis Session

Server-2

Reads Session

Works across multiple instances.

Rate Limiting

API Gateway example.

User
100 Requests/Minute

Store count in Redis.

user123 -> 50 requests

Fast lookup.

Distributed Lock

Scenario:

Two users trying to process same payment.

Payment Service Instance-1
Payment Service Instance-2

Use Redis lock.

LOCK:
transaction:1001

Only one instance proceeds.

Cache Eviction Strategies

When memory is full.

LRU

Least Recently Used

Oldest Unused Data Removed

Most common.

LFU

Least Frequently Used

Least Accessed Data Removed

Redis Cluster

Single Redis:

Redis Server

Problem:

Single Point of Failure.

Redis Cluster:

Redis Node-1

Redis Node-2

Redis Node-3

High Availability.

Common Interview Scenarios

Scenario 1

Database is slow.

Solution?

Answer:

Use Redis cache for frequently accessed data.

Scenario 2

User profile rarely changes.

Solution?

Cache profile in Redis.

Scenario 3

Data changes frequently.

Should you cache?

Answer:

Maybe not.

Cache can become stale.

Need proper invalidation strategy.

Scenario 4

Redis is down.

What happens?

```
Cache Miss
  |
  v
Database
```

Application should fall back to DB.

Scenario 5

Data updated in DB but cache still has old value.

Problem?

```
Stale Data
```

Solution:

```
Java
@CachePut
@CacheEvict
```

Most Asked Interview Questions

What is Redis?

An in-memory key-value datastore used for caching, session storage, rate limiting, and distributed locking.

Why Redis is faster than MySQL?

Because Redis stores data in RAM.

What is Cache Hit?

Data found in cache.

What is Cache Miss?

Data not found in cache, so DB is queried.

What is TTL?

Time after which cached data expires.

What happens if Redis crashes?

Application should fetch data from database.

What is Cache Eviction?

Removing data when cache is full.

Difference Between Redis and Database?

Redis	Database
Memory	Disk
Very Fast	Slower
Temporary Data	Permanent Data
Cache	Source of Truth

Can Redis be used as a Database?

Yes, but in most enterprise microservices:

```
MySQL/PostgreSQL
+
Redis Cache
```

Redis acts as a cache layer.

Interview Answer (30 Seconds)

Redis is an in-memory key-value store used in microservices to reduce database load and improve performance. When a request arrives, the application first checks Redis. If data is available (cache hit), it returns immediately; otherwise (cache miss), it fetches data from the database, stores it in Redis, and returns the response. Redis is commonly used for caching, session management, rate limiting, distributed locks, and improving scalability in high-traffic applications.

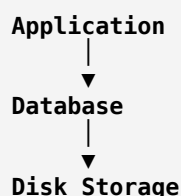
Redis Database

What is Redis Database?

Redis (**RE**mote **DI**ctionary **S**erver) is a **NoSQL in-memory database** that stores data as **key-value pairs**. Unlike MySQL/PostgreSQL, Redis stores most data in **RAM**, making it extremely fast.

Traditional Database vs Redis

MySQL/PostgreSQL

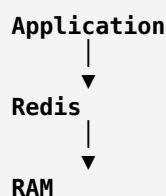


Data is read from disk.

Typical response:

5ms - 100ms+

Redis



Data is read from memory.

Typical response:

< 1ms

Redis Data Model

Redis stores data as:

Key	Value
user:101	Lawrence
user:102	John
user:103	David

Think of it like:

```
Java  
Map<String, Object>
```

stored in memory.

Basic Redis Commands

Store Data

```
redis  
SET user:101 Lawrence
```

Stored as:

```
user:101 -> Lawrence
```

Get Data

```
redis  
GET user:101
```

Output:

```
Lawrence
```

Delete Data

```
redis  
DEL user:101
```

Check Existence

```
redis  
EXISTS user:101
```

Returns:

```
1 = Exists
```

0 = Not Exists

Redis as Database

Redis can act as a primary database.

Example:

```
User Service
  |
  v
Redis Database
```

Store:

```
redis
SET customer:1001 "Lawrence"
```

Retrieve:

```
redis
GET customer:1001
```

Redis Persistence

Many people think Redis loses data after restart.

Not necessarily.

Redis supports persistence.

RDB Snapshot

Redis periodically saves memory to disk.

```
RAM
  |
  v
dump.rdb
```

Example:

```
Save every 5 minutes
```

AOF (Append Only File)

Every write operation is logged.

```
SET user:101 Lawrence
SET user:102 John
```

```
DEL user:101
```

After restart:

Redis replays commands.

More durable than RDB.

Redis Databases

By default Redis has multiple logical databases.

```
DB 0
DB 1
DB 2
...
DB 15
```

Default:

```
DB 0
```

Switch:

```
redis
SELECT 1
```

Redis Data Types

String

Most common.

```
redis
SET user:101 Lawrence
```

Hash

Like Java Object.

```
redis
HSET customer:101
name Lawrence
age 25
city Mumbai
```

Retrieve:

```
redis
```

```
HGETALL customer:101
```

Output:

```
name Lawrence
age 25
city Mumbai
```

List

Like Java List.

```
redis
LPUSH notifications
"Email"
"SMS"
```

Set

Unique values.

```
redis
SADD roles
ADMIN
USER
```

No duplicates.

Sorted Set

Ranking system.

```
redis
ZADD leaderboard
100 Lawrence
90 John
```

Real Spring Boot Example

Suppose:

```
Customer Service
```

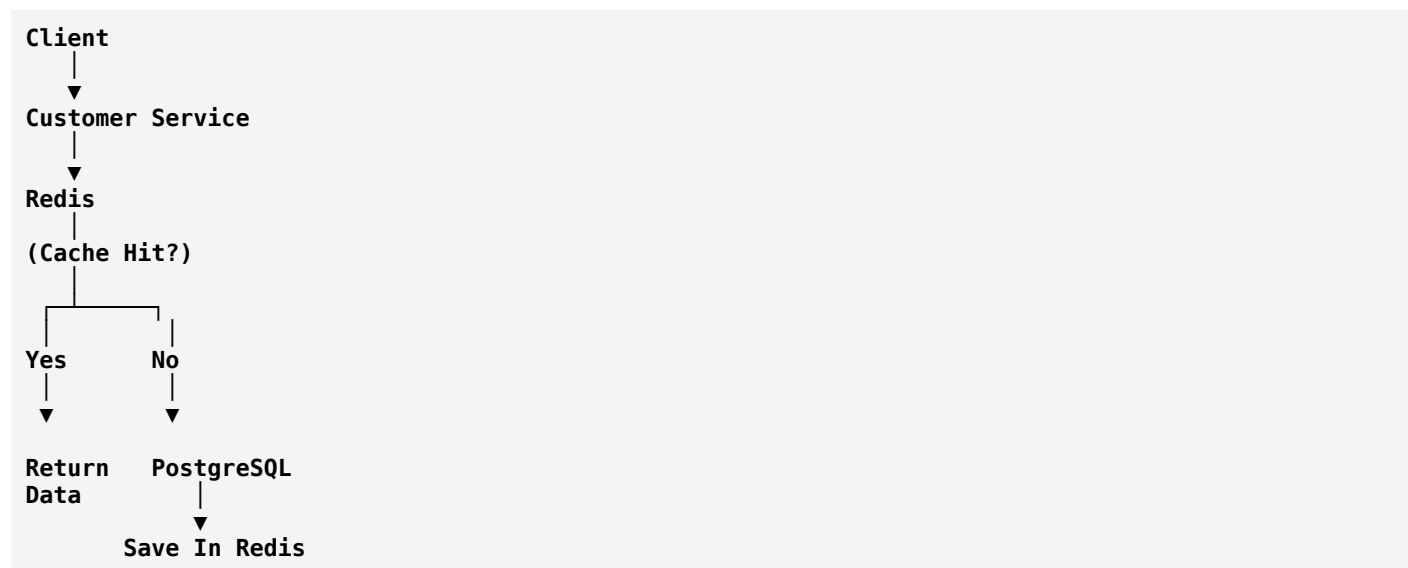
Database:

```
PostgreSQL
```

Redis:

Cache Layer

Architecture:



Why Redis Is Popular in Microservices

Caching

Customer Data
Account Data
Product Data

Session Storage

JWT Sessions
User Login Sessions

Rate Limiting

100 Requests/Minute

Track count in Redis.

Distributed Locking

Transaction Processing

Only one service instance processes it.

Pub/Sub Messaging

Publisher
↓
Redis

↓
Subscribers

Lightweight messaging.

Redis vs MySQL

Feature	Redis	MySQL
Type	NoSQL	Relational
Storage	RAM	Disk
Speed	Very Fast	Slower
Joins	No	Yes
ACID	Limited	Full
Use Case	Cache, Session	Permanent Data

Interview Questions

Is Redis a database?

Yes. Redis is a NoSQL in-memory database.

Why is Redis faster?

Because data is stored in RAM rather than read from disk.

Can Redis persist data?

Yes.

Methods:

- RDB Snapshot
- AOF (Append Only File)

Can Redis replace MySQL?

Usually **No**.

Typical architecture:

MySQL/PostgreSQL
+
Redis

Database remains the source of truth.

Redis improves performance.

Where is Redis mostly used in microservices?

1. Cache
2. Session Storage
3. Rate Limiting
4. Distributed Locks
5. Pub/Sub Messaging

Interview One-Liner

Redis is a high-performance in-memory NoSQL database that stores data as key-value pairs. In microservices, it is primarily used for caching frequently accessed data, reducing database load, improving response times, managing sessions, implementing rate limiting, and distributed locking.

LRU Cache and Interview Questions asked

LRU Cache (Least Recently Used Cache)

LRU is one of the most commonly asked caching interview topics.

What is LRU Cache?

LRU = Least Recently Used

When cache becomes full, remove the item that has not been used for the longest time.

Example:

Cache Size = 3

A
B
C

Now access:

A

Recent order:

B C A

Now insert:

D

Cache is full.

Remove:

B

because it was least recently used.

Final cache:

C A D

Real-Life Example

Think of browser tabs/history.

You frequently visit:

Google
YouTube
ChatGPT

The browser keeps recently used items readily available.

Old unused entries are removed first.

That's the LRU concept.

Why LRU in Redis?

Redis memory is limited.

Suppose:

Redis Memory = 2 GB

Cache becomes full.

Need to remove some keys.

LRU policy removes:

Least Recently Used Keys

to make room for new data.

LRU Example

Cache Size = 4

Initial:

1 2 3 4

Access:

2

Order becomes:

1 3 4 2

Access:

3

Order:

1 4 2 3

Insert:

5

Cache full.

Remove:

1

Result:

4 2 3 5

How LRU is Implemented?

Most efficient implementation:

HashMap + Doubly Linked List

Why?

Need:

1. $O(1)$ Search
2. $O(1)$ Insert
3. $O(1)$ Delete

Architecture

HashMap

```
1 -> Node1
2 -> Node2
3 -> Node3
```

•

Doubly Linked List

```
Head <-> 3 <-> 2 <-> 1 <-> Tail
```

Recent items near Head.

Old items near Tail.

Operations

GET

```
Java
cache.get(2);
```

Move node to front.

Before

```
3 2 1
```

After

```
2 3 1
```

PUT

```
Java
cache.put(4);
```

Insert at front.

```
4 2 3 1
```

If full:

Remove tail.

Java LRU Implementation Using LinkedHashMap

Most asked coding interview solution.

Java

```
import java.util.LinkedHashMap;
import java.util.Map;

class LRUCache<K,V>
    extends LinkedHashMap<K,V> {

    private int capacity;

    public LRUCache(int capacity) {
        super(capacity,0.75f,true);
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(
        Map.Entry<K,V> eldest) {

        return size() > capacity;
    }
}
```

Usage:

Java

```
LRUCache<Integer,String> cache =
    new LRUCache<>(3);

cache.put(1,"A");
cache.put(2,"B");
cache.put(3,"C");

cache.get(1);

cache.put(4,"D");
```

Result:

2 removed

because 2 became least recently used.

LRU vs FIFO

FIFO

Remove oldest inserted item.

1 2 3

Insert 4

Remove 1

Even if 1 was recently accessed.

LRU

Remove least recently accessed item.

```
1 2 3
Access 1
Insert 4
Remove 2
```

Smarter.

LRU vs LFU

LRU

Removes:

```
Least Recently Used
```

LFU

Removes:

```
Least Frequently Used
```

Example:

```
A used 100 times
B used 1 time
```

LFU removes B.

Redis supports both.

Redis Eviction Policies

Common interview topic.

noeviction

```
Memory Full
New Writes Fail
```

allkeys-lru

Remove LRU Key

Most popular.

volatile-lru

Remove LRU Key
Only among keys having TTL

allkeys-lfu

Remove LFU Key

Real Microservice Example

Customer Service:

Customer 101
Customer 102
Customer 103

stored in Redis.

Memory full.

Recently accessed:

101
103

Not accessed:

102

New customer:

104

arrives.

Redis removes:

102

because it is least recently used.

Most Asked Interview Questions

What is LRU Cache?

A cache eviction strategy that removes the least recently used item when cache capacity is reached.

Why is LRU popular?

Because recently used data is likely to be used again.

This improves cache hit rate.

How is LRU implemented?

Using:

```
HashMap
+
Doubly Linked List
```

Time Complexity of LRU Operations?

Operation	Complexity
Get	$O(1)$
Put	$O(1)$
Remove	$O(1)$

Why not use ArrayList?

Deletion in middle:

```
0(n)
```

Too slow.

Difference Between LRU and LFU?

LRU	LFU
Least Recently Used	Least Frequently Used
Tracks recent access	Tracks access count
Simpler	More complex

Which Redis Eviction Policy is Most Common?

```
allkeys-lru
```

When is LRU a Bad Choice?

Suppose:

Data accessed every hour

but not recently.

LRU may evict useful data.

LFU may work better.

Common Coding Interview Question

Design an LRU Cache

Requirements:

```
Java  
put(key, value)  
get(key)
```

Both should run in:

$O(1)$

Expected Answer:

```
HashMap  
+  
Doubly Linked List
```

This is one of the most famous coding questions asked by companies like Amazon, Google, and Microsoft.

30-Second Interview Answer

LRU (Least Recently Used) is a cache eviction algorithm that removes the item that has not been accessed for the longest time when the cache reaches its capacity. It is commonly implemented using a HashMap and a Doubly Linked List to achieve $O(1)$ get and put operations, and it is widely used in Redis caching systems through the **allkeys-lru** eviction policy.